ESP32_UniClock v3.1 program használata

Ebben a dokumentumban megpróbálom összefoglalni az általam fejlesztett ***UniClock*** program működését, telepítését, adaptálását. A program már évek óta fejlődik, köszönhetően *Unferdorben Zoltánnak*, aki mindig új ötletekkel és remek óra panelekkel állt elő. ☺ És tesztelte a folyamatosan fejlődő és mindig új hibákkal rendelkező programverziókat.
Valamint az utóbbi időben *Pintér Sándor* is beszállt a projektbe remek ötletekkel, neki köszönhető a hőmérséklet páratartalom mérés továbbfejlesztése és a Neopixeles led világítás funkcionalitásának kitalálása, valamint végtelenül precíz és szorgalmas tesztelése. ☺
És *Gautier Bálintnak* köszönhető az új alapokra helyezett web oldal.
Az interneten létezik más hasonlóan Uniclock néven szereplő moduláris Nixie óra panel, ennek semmilyen köze sincsen ehhez a projekthez. Sajnálatos névegyezés.

Az UniClock egy Arduino környezetben működő univerzális óra program, amely ESP8266 és ESP32 mikrokontrolleren alapulva Nixie, VFD, Numitron csöves és LED-es kijelzőkkel is működik.
Az óra a pontos időt WiFi-n keresztül az internetes timeserverről, GPS modulról vagy saját DS3231 RTC modulról szinkronizálva biztosítja. Az óra lehet 4, 6 vagy 8 digites, tíz lépésben állítható nappali és éjszakai fényerővel, számjegy animációval, hőmérséklet és páratartalom szenzorokkal, külső fényerő és radarszenzorral, stb. Teljesen önálló, internet független működés is lehetséges, nyomógombos óra/perc beállítással és RTC modullal.

Az óra működési paramétereit saját web oldalán keresztül lehet beállítani. Amennyiben nincsen folyamatos wifi kapcsolat, akkor az óra saját AP-ként működik, így biztosítja a web oldala elérhetőségét. Lehetőség van csövenkénti alá és háttérvilágításra Neopixel WS2812 címezhető ledekkel, választható animációval, fényerővel, stb.
Alarm funkció piezo hangjelzővel, opcionális leállító gombbal.

A github oldal tartalma:
A **https://github.com/gpeter62/ESP_UniClock** *oldalon számos bevált órának megtalálható a kapcsolási rajza és panelterve, Unferdorben Zoltánnak köszönhetően a „/schematics" alkönyvtárban. (Igény esetén gyári panelek és kész órák is hozzáférhetőek.)*

Vannak nyilvánosan elérhető internetes óra projektek, melyek szintén működtethetőek ezzel a programmal. Ezek közül kettő szintén megtalálható a kapcsolások között.

Letölthető sok adatlap a „/Datasheets" alkönyvtárból, amelyeket az órákban szoktunk használni.

Az „/old_versions" alatt korábbi, stabil program verziók találhatóak zip fájban tömörítve.

Felhasználói leírás - az óra működése

Az óra bekapcsolás után elszámol az összes csövön 0-tól 9-ig.

- Önálló működésű óra esetében (GPS vagy RTC szinkron)
Az óra azonnal létrehoz egy ***UNICLOCK*** nevű saját wifi hálózatot. (Vagy ami meg lett adva AP_NAME-ben.)
Erre a hálózatra bármikor fellépve például egy mobiltelefonnal azonnal be lehet jelentkezni az óra web oldalára, melynek címe 192.168.4.1, induláskor ezt ki is írja a kijelzőre.

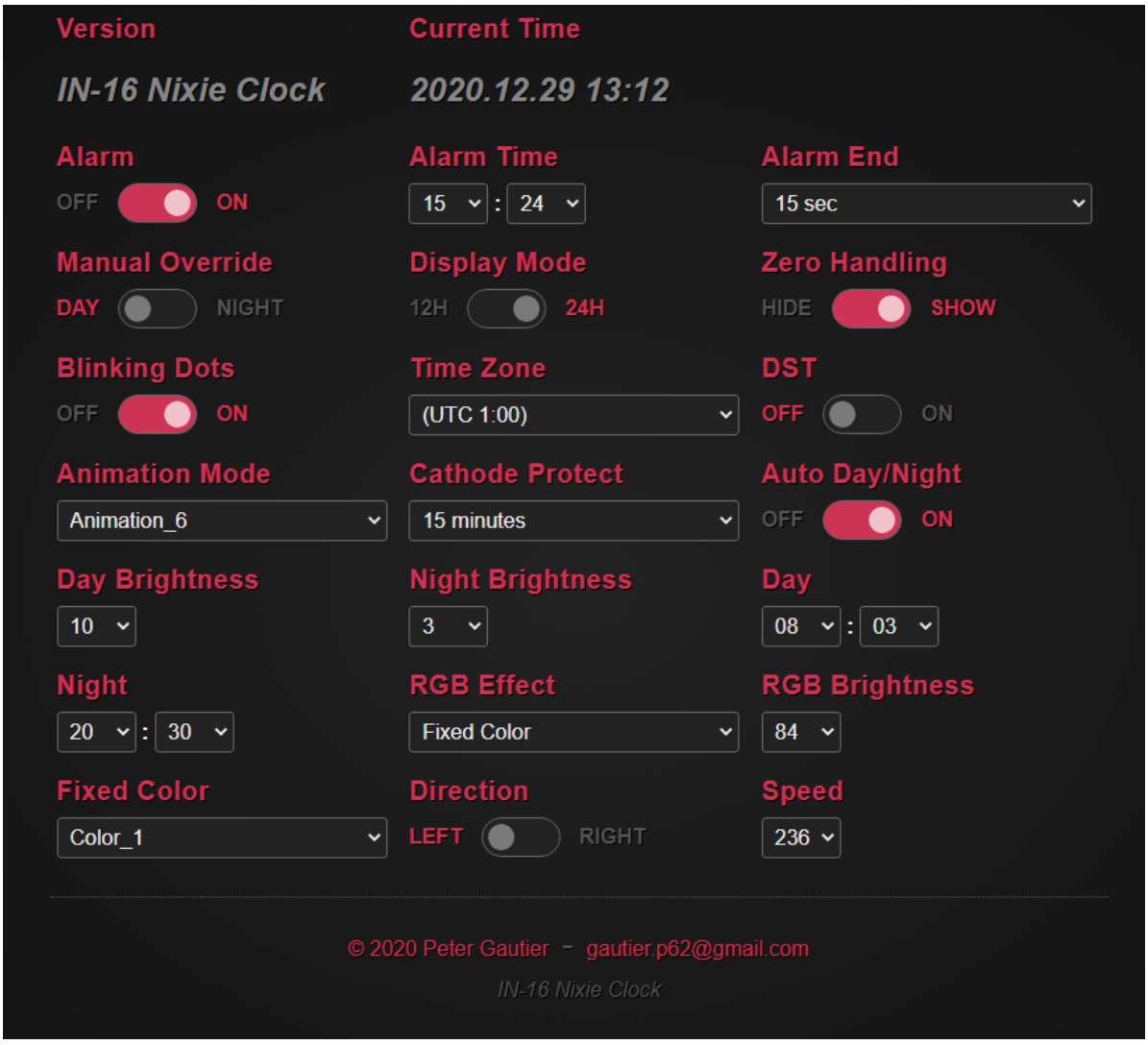
- WiFi-s szinkonizálású óra esetében

Megpróbál fellépni az utoljára használt wifi hálózatra a korábban beállított jelszóval. Ha ez nem sikerül, akkor létrehoz egy ***UNICLOCK*** nevű saját wifi hálózatot. Erre fellépve például egy mobil telefonnal bejön egy web oldal, ahol ki lehet választani az elérhető wifi hálózatok közül, amit használni szeretnénk és megadhatjuk a jelszót. Ez tárolásra kerül és legközelebb már magától fellép ide. (Wifimanager funkció)

Ezután az óra fellép a kiválasztott hálózatra a megadott jelszóval. ha sikeres, a kijelzőre kiírja a kapott IP címét. (például 192.168.1.106)

Az óra web oldala ezen a címen érhető el bármilyen eszközről, amely ugyanezt a wifi hálózatot használja.

Az óra web oldala



Nappali módban minden funkció működik. Az éjszakai módban kikapcsol a led világítás, az elválasztó pont villogtatása és átvált az óra éjszakai fényerőre.

Current time: pontos idő.
Temperature: Amennyiben van hőmérő, akkor a hőmérséklet, amúgy nem látható.
Humidity: Amennyiben van páratartalom mérés, akkor a páratartalom, amúgy nem látható.

Alarm: bekapcsolva az adott időpontban, maximum a magadott időtartamig ébreszt. Amennyiben van kikapcsoló gomb, akkor azzal bármikor leállítható. Amennyiben van Neopixel led az órában, akkor ezeket fehér színben nagy fényerővel villogtatja. Amennyiben van piezo hangjelző, akkor az egyre gyorsabb pittyegéssel jelez.

Manual Override: Kézi átkapcsolás az éjszakai és nappali mód között.
Display Mode: 12/24**:** 12 vagy 24 órás kijelzési mód.
Zero Handling: 12 órás módban mutassa-e a nullát az óra tízes számjegyen.
Blinking Dots: Engedélyezi az elválasztó pont villogtatását. Különben folyamatosan világít.
Time Zone: Időzóna. Magyarországon „1”
DST: Nyári időszámítás bekapcsolása. (Szerencsére többé nem fog kelleni…)
Animation mode: a kijelzőn animálja a számjegyeket 5 különböző módon.
0 = kikapcsolva, 1-5 = animációs módok, 6 = mindig véletlen animáció az 1-5 közül.
Cathode Protect: a megadott időközönként körbefuttatja a kijelzőn a számokat. Ez a Nixie órák esetén fontos, hogy megvédje a csöveket a katód mérgezéstől. Egyébként pedig egy látványos effekt.

Auto Day/Night: engedélyezi az automatikus váltást az éjszakai és a nappali üzemmód között.
Day Brightness: nappali fényerő (1-10 között).
Night Brightness: éjszakai fényerő (0-10 között). Ha nulla, teljesen ki van kapcsolva a kijelző.
Day és ***Night***: a nappal és az éjszaka kezdő időpontja.

Amennyiben van neopixel led világítás, akkor megjelenik az oldalon:

RGB Effect: a Neopixel led animáció mód, jelenleg „OFF” állapot és 10 féle animáció választható.
RGB Brightness: fényerő
Fixed Color: (0-255) amennyiben fix szín „animáció”van kiválasztva, akkor csak ez a szín fog világítani.
Direction Left/Right: a futó animáció iránya
Speed: az animáció sebessége

A színek 0..255 között a színkör alapján működnek, a 192 a fehér szín.

A program paraméterezése és feltöltése a mikrokontrollerre:

A környezet telepítése a szokásos módon történik:

Az Arduino IDE fejlesztő környezet sikeres telepítése után még telepíteni kell az „Eszközök/Alaplap kezelő”-ben a megfelelő ESP8266 vagy ESP32 alaplapokat (ehhez be kell állítani a szokott módon az ArduinoIDE beállítások között az alaplapkezelők megfelelő web címeit.). Ki kell választani a használt alaplapot. A CPU frequency 160MHz és a Flash Size beállításnál legalább 160kbyte FS-t engedélyezni kell az SPIFFS partíciónak.

Telepíteni kell az „***ESPxx Sketch DataUpload***” feltöltő programot, ami a /data könyvtárból fel tudja tölteni az SPIFFS fájlrendszerre a web oldalhoz szükséges fájlokat. Ezek szintén elérhetőek a githubon is az „SPIFFS_uploaders” könyvtárban. (Általában a C:\Programfájlok (x86)\Arduino\Tools alá kell ezeket bemásolni.)

Ennek módját ez a leírás bemutatja:
8266 esetén: https://randomnerdtutorials.com/install-esp8266-filesystem-uploader-arduino-ide/
ESP32 esetén: https://randomnerdtutorials.com/install-esp32-filesystem-uploader-arduino-ide/

A UniClock programot a szokott módon az Arduino könyvtárba kell telepíteni. (Általában Sajátgép/Dokumentumok/Arduino helyen található.)
A programkönyvtár nevét át kell nevezni ESP32_UniClock2 –re, hogy megegyezzen a főrpogram nevével.
A működéshez szükséges library-ket szintén a zip fájlokból kibontva a szokásos módon az Arduino/libraries alá kell telepíteni. (Ez általában Sajátgép/ Dokumentumok/Arduino/libraries helyen található.) A program futásához szükséges library-k megtalálhatóak az UniClock github /libraries könyvtárban, tömörített formában.

A program beállítása az óra hardverhez:

A ***clocks.h*** fájlban fordítás előtt az alábbi paramétereket lehet a használt hardver ismeretében beállítani, ***a főprogramban és az alprogramokban általában már nem kell semmit beállítani***. Amelyik paraméter nem került beállításra egy óránál, az alapértelmezés szerint ki van kapcsolva illetve egy alapértelmezett értékre áll be. Tehát minden óránál csak azt kell beállítani, ami ott konkrétan használva van a hardverben, a nem használt opciókat nem kell mindig beállítgatni.

```
/*_____________________________ USABLE PARAMETERS ____________________________________

  //#define DEBUG  //Enable Serial Monitor, 115200baud (only, if TX pin is not used anywhere!!!)
  //--------------------------- CLOCK EXTRA OPTION PARAMETERS -------------------------------
  //#define USE_DALLAS_TEMP //TEMP_DALLAS_PIN is used to connect DS18B20 temperature sensors
  //#define USE_DHT_TEMP    /TEMP_DHT_PIN is sensor pin
  //#define USE_BME280      //I2C Temperature + humidity + pressure, SDA+SCL I2C pins are used!
  //#define USE_BMP280      //I2C Temperature + barometric pressure, SDA+SCL I2C pins are used!
  //#define USE_AHTX0       //I2C Temperature + humidity, SDA+SCL I2C pins are used!
  //#define USE_SHT21       //I2C Temperature + humidity, SDA+SCL I2C pins are used!
  //#define USE_RTC         //DS3231 realtime clock, SDA+SCL I2C pins are used!
  //#define USE_GPS         //use for standalone clock, without wifi internet access
  //#define USE_NEOPIXEL   //WS2812B led stripe, for tubes backlight. Don't forget to define
tubePixels[] !
```

Ha a ***DEBUG*** engedélyezve van, akkor a Soros Monitoron a program induláskor mindent kiír, követhető, mi történik. Sebesség: 115200 baud.

USE_DALLAS_TEMP: Dallas DS18B20 hőmérséklet szenzor engedélyezése.
A TEMP_SENSOR_PIN értékét be kell állítani, maximum 2 ilyen szenzor használata támogatott egy közös buszon. (2 szenzor esetén a szenzorok sorrendjét jelenleg csak a dallas_temp.ino modul elején lehet beállítani.)

USE_DHT_TEMP: DHT11, DHT21, vagy DHT22 hőmérő és páratartalom szenzor használható.
#define DHTTYPE DHT22 sorban adható meg a szenzor pontos típusa.
A TEMP_DHT_PIN értékét be kell állítani, maximum 1 ilyen szenzor használata támogatott.

USE_BME280, USE_BMP280, USE_AHTX0, USE_SHT21: I2C buszos szenzorok használatához az SDA és SCL lábakat be kell állítani, ez bármely szabad digitális láb lehet.

USE_RTC: Ha RTC engedélyezve van, akkor DS3231 RTC modult fog használni, I2C buszon. (Az I2C busz PIN_SDA és PIN_SCL GPIO lábakat definiálni kell a clock.h fájlban.) Az rtc.ino modul be lesz kapcsolva, itt adható meg a nyomógombok és az üzemmód kapcsoló GPIO lába. RTC esetén a GPS nem választható. Ez a mód az üzemmód választó kapcsoló alapján működhet wifi óraként is, frissítve az RTC időt az internetről vagy önálló óraként is, csak az RTC-re támaszkodva. Ekkor kézzel beállítható a pontos idő a nyomógombokkal.

USE_GPS : Ha a GPS engedélyezve van, akkor nem fog wifi hálózatot keresni, mindenképpen a GPS-ről szinkronizálja az órát, ez önálló működésű üzemmód. (***gps.ino*** modul bekapcsolásra kerül.) Jelenleg a GPS paramétereit is itt kell beállítani, amennyiben eltérnek a szokásostól.

USE_NEOPIXEL : WS2812 Neopixel ledek használhatóak. (Makuna-féle library van használva, ez a *z_neopixel.ino* modul)
A modulban a tubePixels[] tömbben kell definiálni, hogy a ledsor egyes pixelei melyik csőhöz tartoznak. Lehet, hogy csöveként csak egy led szükséges, de lehet több led is csövenként.
8266 esetében csak az RX láb használható vezérlésre, ESP32 esetén bármelyik 32-nél kisebb GPIO láb alkalmas.

Kijelző meghajtó modul kiválasztása:
Az ESP32 lapok esetében az alábbi meghajtó típusból választhatunk jelenleg:

- #define MULTIPLEX74141_ESP32 // *ESP32 Nixie driver*
- #define MAX6921_ESP32 // ESP32 7-segment VFD driver
- #define NEWHV5122 // 4..8 tubes Nixie driver (current version; plain HV5122/HV5122.ino is legacy)
- #define PT6355 //VFD clock, multi-segment tubes

8266 esetében az alábbi meghajtó programok állnak rendelkezésre:

- #define MULTIPLEX74141 // *ESP32 Nixie driver*
- #define MAX6921 // ESP32 7-segment VFD driver
- #define NO_MULTIPLEX74141 //4..6 Nixie tubes, serial latch driver, 74141 for each tube
- #define MM5450 // *6..8 LEDs*
- #define MAX7219CNG // *4..8 LEDs*
- #define Numitron_4511N // 4x *Numitron tube clock*
- #define SN75512 // *4..8 VFD tubes*
- #define Samsung // Samsung serial display (jelenleg nem elérhető – nincs a build része)
- #define PCF_74141 // pin expander tube selection

További kijelző-driverek (a v2.5 óta bekerültek)

A fejlesztés a kézikönyv eredeti (v2.5-ös firmware-hez írt) megírása után is folytatódott. Az alábbi driverek is elérhetők mára – a megfelelő #define sort kell kikommentezni a clocks.h-ban, a hardvernek megfelelő lábkiosztással együtt:

- PT6311 – Princeton Technology „okos” VFD-vezérlő, saját beépített karakter-RAM-mal és scan-generátorral (ESP8266). Például az IV-22-es „orosz órához” használt.

- SN75518_ESP32 – Texas Instruments 32-kimenetű VFD grid/szegmens-vezérlő (ESP32). Támogatja mind a 7-szegmenses, mind a 16-szegmenses VFD-cső bekötést.

- SN75512 – Texas Instruments VFD-vezérlő, 2 chip kaszkádolva.

- NEWHV5122 – a jelenlegi, aktívan karbantartott HV5122 Nixie-driver (2 kaszkádolt HV5122 chip, 64 kimenet). Ez váltja fel a fentebb látható, régi, mára már nem használt sima HV5122 makrót.

- ZM1500_ESP32 – 4 kaszkádolt 74HC595 shiftregiszter + optocsatolók (csak ESP32).

- MULE_V2 – CD4543 + CD4028 alapú 7-szegmenses LED-driver a „Mule V2” board-hoz (csak ESP32).

- VQC10 – 5×7 pontmátrixos VQC10 kijelzőelemeket vezérel; a fenti cső-driverekkel ellentétben teljes ASCII szöveget is meg tud jeleníteni, nem csak számjegyeket.

- WORDCLOCK – egészen más kijelzési stílus: csövek helyett egy WS2812 NeoPixel-mátrix (lézervágott vagy nyomtatott panel mögött) egyes betűket világít ki, így az idő szavakban jelenik meg (pl. „NEGYED HÉT”), nem számjegyekben. Angol, német és magyar szólayout is elérhető – lásd lentebb a „Szódíszórák” szakaszt.

- TUBE1CLOCK – egycsöves változat (1tube.ino) nagyon kompakt, egyetlen Nixie-csöves órákhoz.

- DUMMY – egyáltalán nincs fizikai kijelző; csak szenzoradatot (hőmérséklet/páratartalom/stb.) publikáló „óra” MQTT-n keresztül (lásd lentebb).

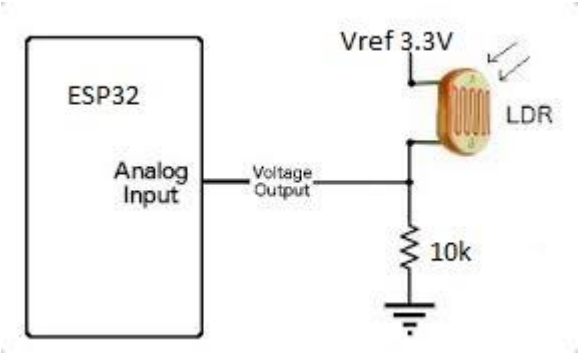
A teljes, mindig naprakész listáért – pontosan mely lábakat és opciókat várja az egyes driver – lásd a docs/DEVELOPER_GUIDE.md fájlt a projekt repository-jában (7. fejezet, „Driver catalog”).

Egyéb vezérlő lábak megadása:
A megadható láb kiosztás értelemszerűen megegyezik mindkét esetben. Ha egy funkció nincsen használva, akkor az értékét -1 –re kell állítani. **Mindig a GPIO számot célszerű megadni, ez nem függ a modulok típusától!!!**

```
//--------------------- PINOUT & PIN PARAMETERS -----------------------------------------
  //#define PIN_SDA xx             // you can set the used SDA and SCL pins
  //#define PIN_SCL xx             // if it is not default value
  //#define COLON_PIN   -1        //Blinking Colon pin.  If not used, SET TO -1
  //#define TEMP_DALLAS_PIN -1     //Dallas DS18B20 temp sensor pin.  If not used, SET TO -1
  //#define TEMP_DHT_PIN -1       //DHT temp sensor pin.  If not used, SET TO -1
  //#define DHTTYPE DHT22         //DHT sensor type, if used...
  //#define LED_SWITCH_PIN -1     //external led backlight ON/OFF.  If not used, SET TO -1
  //#define DECIMALPOINT_PIN -1   //Nixie decimal point between digits. If not used, SET TO -1
  //#define ALARMSPEAKER_PIN -1   //Alarm buzzer pin
  //#define ALARMBUTTON_PIN -1    //Alarm switch off button pin
  //#define ALARM_ON HIGH         //HIGH or LOW level is needed to switch ON the buzzer?
  //#define NEOPIXEL_PIN 3        //8266 Neopixel LEDstripe pin is always the RX pin!!!
  //#define RGB_MIN_BRIGHTNESS 8   //Neopixel leds minimum brightness
  //#define RGB_MAX_BRIGHTNESS 255 //Neopixel leds maximum brightness
  //#define NEOPIXEL_PIN 2        //8266 Neopixel LEDstripe pin is always the RX pin!!!
```

```
  //#define RADAR_PIN 34          //Radar sensor pin
  //#define RADAR_TIMEOUT 300     //Automatic switch off tubes (after xxx sec
  //#define TUBE_POWER_PIN -1     //Filament or HV switch ON/OFF pin
  //#define TUBE_POWER_ON HIGH    //HIGH or LOW level is needed to switch ON the TUBE POWER?
  //#define LIGHT_SENSOR_PIN -1   //Environment light sensor, only ADC pins are usable! ESP32:
34,35,36,39... 8266: only A0
  //#define REF_RESISTANCE    10000.0   // Resistor value is 10k, between LDR sensor and GND
  //#define MAXIMUM_LUX 100                  //Lux level for maximum tube brightness
  //#define LUX_CALC_SCALAR   12518931 * 1.2  //Calibrate here the LDR sensor
  //#define LUX_CALC_EXPONENT -1.405          //LDR sensor characteristic
```

A ***PIN_SDA*** és ***PIN_SCL*** definiálja az I2C busz által használható lábakat, bármely digitális láb megfelelő.
A ***COLON_PIN*** az óra és perc csövek között villogó pontokat kapcsolja. („***Blinking Dots***”) Nixie csöveknél szoktuk használni, ha nincsen tizedes pont a csőben.
A ***LED_SWITCH_PIN*** egy kapcsoló láb, amely nappal bekapcsol, éjjel kikapcsol. Bármilyen külső vezérlésre felhasználható.
A ***NEOPIXEL_PIN*** vezérli a WS2812 ledeket, amennyiben van ilyen installálva. Ez 8266 esetén kizárólag az RX láb lehet (8266 belső felépítése miatt), ESP32 esetén bármely digitális láb lehet. Az ***RGB_MIN_BRIGHTNESS*** és ***RGB_MAX_BRIGHTNESS*** megadja a lehetséges szélső értékeit a fényerőnek.
A ***RADAR_PIN*** külső mozgásérzékelő vagy radar modul kapcsoló jelét fogadja. Amennyiben nincsen mozgás, akkor ***RADAR_TIMEOUT*** másodperc múlva lekapcsolja a kijelzést. Amennyiben a ***TUBE_POWER_PIN*** is be van kötve, akkor ilyenkor lekapcsolódik a cső fűtése is.
A ***LIGHT_SENSOR_PIN*** analog bementként van használva az alábbi ábra szerint. (A referencia feszültség ESP32 esetén 3.3Volt, az 8266 chip esetén csak 1Volt, mivel az ADC csak 1 Voltig mér. De a legtöbb 8266 board már tartalmaz egy feszültségosztót, így itt is tudunk 3 Voltig mérni.) Amennyiben eza szenzor beépítésre kerül, akkor lehetőség van automatikus fényerőszabályzós kijelzésre a nappali időszakban.
A program tartalmaz egy Luxmérő algoritmust, ennek paraméterei is itt adhatók meg.
A szokásos irodai fényerő 250 Lux körül van, egy szoba ennél általában jóval sötétebb. A ***MAXIMUM_LUX*** értéke megadja, hogy mekkora fényerőnél érje el a kijelzés a maximális fényerőt, ennek alapértelmezett értéke 100 Lux. A fényerőszabályozás teljesen folyamatos, nem a szokott fényerő lépcsőben történik és a 1..10 fényerő értékek között működik. A LED világítás fényereje is folyamatosan szabályozódik, a web oldalon beállított maximális RGB fényerő és a ***RGB_MIN_BRIGHTNESS*** között.



A többi pin funkciója remélhetőleg értelemszerű mindenkinek.

#define MAXBRIGHTNESS 10
Ez a csövek maximális fényereje, nem szabad módosítani.

A következő blokkban azt adhatjuk meg, hogy az óra egy percen belül hányadik másodpercnél melyik adatot írja ki a kijelzőre. (TEMP_START … DATE_END közötti paraméterek)
Az ***ENABLE_CLOCK_DISPLAY*** segítségével letiltható a teljes idő és dátum kijelzés, ennek csak akkor van értelme, ha

kizárólag hőmérséklet és páratartalom mérőt szeretnénk óra funkció nélkül.
A SHIFT_TUBES_LEFT_BY_1 paraméter 1 digittel balra eltolja a kijelzést, hőmérős óráknál.
Az ***ANIMSPEED*** érték a kijelző áttűnési animációk sebességét határozza meg.
A ***TEMP_CHARCODE***, a ***GRAD_CHARCODE*** és a ***PERCENT_CHARCODE*** értéke a Celsius, a fok és a „%” karakter kódját határozza meg. 7 szegmenses csöveknél ez a szokásos érték. Amennyiben speciális karakteres Nixi csövet használunk, akkor azt a számot kell beírni, amelyik számjegy helyén az adott jelzés található.

```
//Display temperature and date in every minute between START..END seconds
  //Display temperature and date in every minute between START..END seconds
  //#define ENABLE_CLOCK_DISPLAY true  //false, if no clock display is needed (for example:
thermometer + humidity only)
  //#define SHIFT_TUBES_LEFT_BY_1   //shift leftIP address by 1 tube the display, if a
thermometer is used with spec tube
  //#define LEFTDECIMAL false      //set true (Z574M), if decimal point is on the left side on
the tube. Else set false (Z573M)!
  //#define TEMP_START  35        //Temperature display start..end
  //#define TEMP_END    40
  //#define HUMID_START 40        //Humidity% display start..end
  //#define HUMID_END   45
  //#define DATE_START  45        //Date is displayed start..end
  //#define DATE_END    50
  //#define ANIMSPEED   50        //Tube Animation speed in millisec
  //#define TEMP_CHARCODE 15      //Thermometer "C", set to -1 to disable
  //#define GRAD_CHARCODE 16      //Thermometer grad, set to -1 to disable
  //#define PERCENT_CHARCODE 17   //Humidity %
  //#define DOUBLE_BLINK          //both separator points are blinking (6 or 8 tubes VFD clock)

A Neopixel ledeknél a megengedett minimum és maximum fényerőt így adhatjuk meg a clocks.h-ban:
  #define RGB_MIN_BRIGHTNESS 8   //Neopixel leds minimum brightness
  #define RGB_MAX_BRIGHTNESS 255 //Neopixel leds maximum brightness

A következő paraméter blokkban a wifi kezelés adatait adhatjuk meg:
  //#define AP_NAME "UNICLOCK"   //Access Point name
  //#define AP_PASSWORD ""       //AP password
  //#define WEBNAME "LED UniClock"      //Clock's name on the web page
```

AP_NAME a létrehozandó AccessPoint (röviden AP) neve
AP_PASSWORD az AP eléréséhez szükséges jelszó. Jelenleg nincsen beállítva, nyitott az AP.
WEBNAME az weboldalon kiírt név, szabadon megadható (pl. „Pistike órája”)
További testreszabási lehetőségek a főprogramban:
A factoryReset() programrészben adjuk meg a gyári alapállapothoz tartozó értékeket.

WiFi web oldal módosítása:
A „/data” könyvtárban találhatóak a web oldal fájlai. Azt az SPIFFS uploaderrel kell feltölteni az ESP modulra.

Ha minden sikerült, feltöltés után érdemes bekapcsolni a Soros monitort, itt induláskor mindent kiír az óra, hogy mi történik és kiírja azt is, ha valahol hiba van. Például:

```
Starting ESP32 IN-11 UniClock 3.1
- ALARMSPEAKER_PIN: GPIO33
- RADAR_PIN: GPIO21
- LIGHT_SENSOR_PIN: GPIO35
- TEMP_DALLAS_PIN: GPIO23
DS18B20 sensors found:1
Temperature requested
Setup MAX6921 pins for VFD Clock...
- PIN_LE: GPIO14
```

```
- PIN_BL: GPIO12
- PIN_DATA: GPIO27
- PIN_CLK: GPIO13
--- Generating segment pins bitmap ---
--- Generating digit pins bitmap ---
---- Generated Character / Pins table -----
Number of digits:6
Disable tubes
Loading setting from EEPROM. EEPROM ver:200
Enable tubes:0
Setup NeoPixel LEDS
- NEOPIXEL_PIN: GPIO2
Pixel count: 10
Brightness:8 - 255
___ USED CLOCK PINS ___
2: NEOPIXEL_PIN
12: PIN_BL
13: PIN_CLK
14: PIN_LE
21: RADAR_PIN
23: TEMP_DALLAS_PIN
27: PIN_DATA
33: ALARMSPEAKER_PIN
35: LIGHT_SENSOR_PIN
_________________________
```

A legfontosabb két univerzális kijelző meghajtó program beállítása:

multiplex74141.ino és multiplex74141_ESP32.ino
A program multiplex módon egy darab 74141 segítségével hajt meg tetszőleges számú Nixie csövet.

Értelemszerűen meg kell adni a csövek engedélyező lábait kapcsoló GPIO számokat:
byte digitEnablePins[] = {14,12,13,15};

És a 74141 ABCD vezérlő lábait vezérlő GPIO számokat: byte ABCDPins[4] = {2,4,5,0};

A maxDigits egy nagyon fontos érték, ez a csövek darabszáma, ezt a program innen fogja megtudni, kézzel nem szabad változtatni! const int maxDigits = sizeof(digitEnablePins);

int PWMrefresh=11000; //msec, Multiplex time period. Greater value => slower multiplex freq
int PWMtiming[MAXBRIGHT+1] = {0,1000,2000,3000,4000,5000,6000,7000,8000,9000,10000};

A ***PWMrefresh*** egy darab cső multiplex időállandója.
A ***PWMtiming[]*** az egyes 0..10 fényerőértékekhez tartozó időállandó.
A program a fényerőhöz tartozó értékig kigyújtja a csövet, majd a
PWMrefresh - PWMtiming[fényerő] ideig kikapcsolva tartja.

(Amennyiben van LIGHT_SENSOR installálva, akkor az ***autoBrightness*** változó hatására folyamatos fényerő változtatás történik, de hasonló módon.)
Majd lép tovább a következő csőhöz.
Látható, hogy a PWMrefresh értéke nagyobb kb.10%-kal, mint a legnagyobb fényerőhöz tartozó időzítés. Ez azért van, hogy a mindenképpen stabilan kikapcsoljon a cső és ne legyen ghost effektus.
Amennyiben a hardver tranzisztoros meghajtású, akkor ez a különbség lehet minimális, de optocsatolós meghajtásnál ez feltétlenül szükséges, mert az elég lassan olt ki.
A csövek számától függően szükséges lehet az időzítések változtatása, hogy sok csőnél se legyen vibrálás. Illetve, bárki ízlése szerint szabadon beállíthatja, hogy mekkora fényer értékek tartozzanak az egyes fényerő szintekhez.

MAX6921.ino és MAX6921_ESP32.ino
A program egy darab MAX6921 IC segítségével hajt meg tetszőlegesen bekötött VFD kijelzőket.
A ***segmentEnablePins[]*** tömbben meg kell adni, hogy az MAX6821 melyik OUT lábain találhatóak az a,b,c,d,e,f,g,h szegmensek. Mindig kötelező 8 értéket megadni!!! A „h” szegmens értelemszerűen a tizedespont.
digitEnablePins[] tömbben meg kell adni, hogy melyik OUT lábak kapcsolják a digiteket. Annyi lábat kell megadni, ahány számjegy van. (Ez a példa egy IV18 hosszú, sok számjegyes csőhöz készült.)
A MAX6921 pins szekcióban értelemszerűen az IC-t vezérlő ESP GPIO lábai vannak megadva.

//Fill this table with the OUT positions of the MAX6921 chip!
const byte segmentEnablePins[] = {0,2,5,6,4,1,3,7}; //segment enable OUTbits of MAX6921 (a,b,c,d,e,f,g,DP)
const byte digitEnablePins[] = {18,11,17,12,16,13,14,15}; //digit enable OUTbits of MAX6921

```
//MAX6921 pins
 #define PIN_LE    12  // D6 Shift Register Latch Enable
 #define PIN_CLK   13  // D7 Shift Register Clock
 #define PIN_DATA  14  // D5 Shift Register Data
 #define PIN_BL    15  // D8 Shift Register Blank (1=display off     0=display on)
```

A charDefinition[] táblázat értelemszerűen a karakterkészletet definiálja. Itt látható, hogy korábban miért a #define TEMP_CHARCODE 15 és a #define GRAD_CHARCODE 16 határozta meg Celsius és % jeleket. ☺

Az alábbi értékek a ***multiplex74141.ino*** modulnál leírt logikával működnek.

Újdonságok a v2.5 óta (2026-os, v3.1 firmware-hez készült frissítés)

Az alábbi szakaszok olyan funkciókat mutatnak be, amelyek a kézikönyv eredeti megírása után kerültek a firmware-be, a kódbázis 2026-os teljes körű átvizsgálása alapján.

Home Assistant / MQTT integráció

Az óra képes a mért szenzoradatait (bizonyos konfigurációknál pedig egy másik óra adatait is) egy MQTT brókernek publikálni Home Assistant automatikus felismeréshez. Engedélyezése a clocks.h-ban: #define USE_MQTT. A bróker címét, felhasználónevét és jelszavát az óra saját weboldaláról lehet beállítani – a firmware gyárilag egy alapértelmezett bróker-címmel és alapértelmezett mqtt/mqtt jelszóval indul (a factoryReset()-ben), ezért mindenképp állítsd át a saját brókeredre, mielőtt egy MQTT-s órát nem teljesen megbízható hálózatra kötsz.

Master/slave szenzor-megosztás: egy fizikai hőmérséklet-/páratartalom-/radarszenzorral rendelkező óra „master”-ként publikálhatja a méréseit MQTT-n; más órák (amelyeknek esetleg nincs is saját szenzoruk) „slave”-ként állíthatók be a USE_MASTER_CLOCK (és szükség szerint a USE_MASTER_TEMP / USE_MASTER_HUMID / USE_MASTER_RADAR) bekapcsolásával a saját clocks.h-jukban – így egy háztartáson belül több óra is megoszthat egyetlen fizikai szenzort.

További I2C szenzorok

A fentebb már leírt DS18B20/DHT szenzorokon és a DS3231 RTC-n kívül az alábbi I2C szenzorok is támogatottak, és szabadon kombinálhatók a közös I2C buszon (PIN_SDA/PIN_SCL):

- BME280 (USE_BME280) – hőmérséklet + páratartalom + légnyomás

- BMP280 (USE_BMP280) – hőmérséklet + légnyomás, páratartalom nélkül

- AHT10/AHT20 (USE_AHTX0) – hőmérséklet + páratartalom

- SHT21 (USE_SHT21) – hőmérséklet + páratartalom

- BH1750 (USE_BH1750) – dedikált I2C fényerősség-/lux-szenzor, alternatívája a LIGHT_SENSOR_PIN-en lévő analóg LDR-nek, automatikus fényerő-szabályozáshoz

Egyazon órán több szenzortípus is engedélyezhető egyszerre (pl. egy BME280 és egy SHT21 együtt) – minden mérés saját helyet kap a weboldalon és a csöveken megjelenő körforgásban.

Szódíszórák

Egy egészen más kijelzési stílus is elérhető: Nixie/VFD/Numitron csövek helyett egy WS2812 NeoPixel-mátrix (lézervágott vagy nyomtatott panel mögött) egyes betűket világít ki, így az idő mondatban jelenik meg (pl. „NEGYED HÉT”), nem számjegyekben. Engedélyezése a clocks.h-ban: #define WORDCLOCK; angol, német és magyar szólayout is elérhető külön példa-konfigurációként. Mivel nincsenek kijelző-csövek, a szokásos cső-specifikus beállítások egy része (animáció mód, katódvédelem, stb.) itt nem értelmezhető.

Gyári alapállapot és firmware-frissítés a weboldalról

Ha a FACTORYRESET_PIN be van kötve az adott órán, a gomb bekapcsolás közbeni nyomva tartásával az összes weboldali beállítás (WiFi, ébresztő, fényerő, RGB, MQTT, stb.) visszaáll a factoryReset()-ben megadott gyári alapértékre. Az óra emellett támogatja a firmware over-the-air (OTA) frissítését is közvetlenül a weboldaláról, számítógéphez való visszacsatlakoztatás nélkül.

Hibajavítások és ismert problémák (2026-os review)

2026-ban egy teljes körű kódátvizsgálás történt, aminek eredményeként több megerősített hiba is javításra került (főként tömbhatár-túllépési/indexelési hibák egyes cső-drivereknél), és egy dokumentált lista készült a fennmaradó ismert problémákról – köztük egy biztonsági megjegyzés az MQTT-modul alapértelmezett bróker-hitelesítő adatairól. A teljes, aktuális listáért lásd a docs/CODE_REVIEW.md fájlt a projekt repository-jában.

További clocks.h paraméterek (a fentiekben nem szereplők)

A kézikönyv elején bemutatott paraméterblokk csak a v2.5-ben már létező opciókat sorolja fel. A clocks.h azóta jelentősen bővült – az alábbi paraméterek is elérhetők (téma szerint csoportosítva). Mint minden clocks.h opciónál, ezek is csak akkor relevánsak, ha a megfelelő funkció/driver be van kapcsolva az adott óránál; a nem használtakkal nem kell foglalkozni.

MQTT / Home Assistant (a koncepcióról lásd fentebb a „Home Assistant / MQTT integráció" részt)

- MQTT_PREFIX – az óra saját publikált szenzor-topicjainak előtag-stringje

- USE_MASTER_CLOCK – egy másik óra publikált szenzoradatára iratkozik fel, a saját (helyi) szenzorok helyett vagy mellett

- USE_MASTER_TEMP / USE_MASTER_HUMID / USE_MASTER_RADAR – mely mérési adatokat vegye át a master órától

- MASTER_TEMPERATURE_TOPIC / MASTER_HUMIDITY_TOPIC / MASTER_RADAR_TOPIC – a pontos MQTT topic-string, amire a master órán fel kell iratkozni

PWM RGB háttérvilágítás (a NeoPixel alternatívája – lásd USE_PWMLEDS)

- PWM1_PIN / PWM2_PIN / PWM3_PIN – a három GPIO láb a Piros/Zöld/Kék PWM kimenetekhez (csak ESP32)

WiFi kiegészítők

- USE_WIFIMANAGER – ha nincs beállítva WiFi hálózat, az ESPAsyncWiFiManager captive-portál beállító folyamatára vált az óra saját, fix AP-ja helyett

- MDNSNAME – az mDNS hosztnév (alapértelmezetten „uniclock"), így az óra elérhető pl. http://uniclock.local címen is, IP-cím beírása nélkül

RTC kézi beállító nyomógombok (lásd USE_RTC)

- PIN_MODE_SWITCH – normál óra mód és időbeállító mód között kapcsol

- PIN_FLD_BUTTON – a szerkesztendő mezők között lép (óra/perc/stb.)

- PIN_SET_BUTTON – növeli az éppen kiválasztott mező értékét

Numitron driver specifikus paraméterek (lásd Numitron_4511N)

- PCB_VERSION – melyik fizikai Numitron panel-revíziót (1, 2 vagy 3) használjuk

- LTBIpin – a közös „Lamp Test / Blanking Input” láb, a teljes panel fényerő-PWM-jéhez

További kijelzőkarakterek (a TEMP_CHARCODE / GRAD_CHARCODE / PERCENT_CHARCODE mellett)

- PLUS_CHARCODE / MINUS_CHARCODE – +/- előjel karakterkód, olyan hőmérő-óráknál, ahol van előjel-digit

- UPPER_CIRCLE_CHARCODE / LOWER_CIRCLE_CHARCODE – egy „°” vagy „%” jel felépítése két félkör-karakterből, ha nincs erre önálló karakter a csövön

Alternatív hőmérséklet/páratartalom kijelzési elrendezések

- DISPLAYTEMP_ONLY_2DIGITS / DISPLAYHUMID_ONLY_2DIGITS – kompakt, csak 2 számjegyes kijelzés az alapértelmezett többjegyű helyett (rövid kijelzőknél hasznos)

- TEMP_START_POSITION / HUMID_START_POSITION – melyik cső-pozíción kezdődjön a kompakt kijelzés

Egyéb

- DATE_REPEAT_MIN – milyen gyakran (percben) jelenjen meg a dátum a normál körforgásban (nem azonos a DATE_START/DATE_END-del, ami a percen belüli másodperc-ablakot adja meg)

- THERMOMETER_CLOCK – 6 csöves órát „előjel, 4 számjegy, C/F egység” elrendezésűnek jelöl, tisztán hőmérő óráknál

- DISABLE_NIGHT_ANIMATION – kikapcsolja a csőváltás-animációt éjszakai módban (nappali fényerőnél továbbra is animál)

- DISABLE_BROWNOUT – kikapcsolja az ESP32 brownout-detektorát – csak néhány panelnél/tápegységnél szükséges, ahol induláskor feszültségesés-probléma van

- FW – a firmware rövid build-azonosító stringje, ebből épül fel az OTA frissítés letöltési URL-je (firmwareServer/FW.bin)

- SIMULATE_TEMPERATURE_SENSOR – csak teszteléshez: valós szenzor nélkül hamis hőmérséklet/páratartalom értékeket generál, a kijelzési logika teszteléséhez

- MAXDIGITS – néhány drivernél közvetlenül ezzel adható meg a csövek száma, a pin-tömbből való automatikus számolás helyett

Szódiszóra (WORDCLOCK) specifikus paraméterek (lásd fentebb a „Szódiszórák” részt)

- WORD_LENGTH – a leghosszabb szó karakterhossza az adott nyelvi elrendezésben

- XMAX / YMAX – a LED-mátrix szélessége/magassága pixelben

- START_CORNER – a LED-mátrix bekötése melyik fizikai sarokból indul (LEFTUP/RIGHTUP/LEFTDOWN/RIGHTDOWN) – ez kell a logikai betűpozíciók helyes fizikai LED-indexre való leképezéséhez

- WEEKDAY_POSX / WEEKDAY_POSY – a hét napjait jelző betűk kezdő pozíciója a mátrixon

- SENSOR_POSX / SENSOR_POSY – a kis állapotjelző pontok (pl. WiFi kapcsolat) pozíciója a mátrixon

A driver-specifikus GPIO lábnevek (pl. PIN_CS/PIN_SIN a PT6355-nél, PIN_DIN/PIN_LOAD a PT6311-nél, PIN_STB az SN75518-nál, dataPin/clkPin/latchPin a shiftregiszteres drivereknél, stb.) driverenként eltérők – ezeket érdemesebb a legközelebbi példa CLOCK_NN blokkból átmásolni, mint nulláról beírni; a teljes referencia a docs/DEVELOPER_GUIDE.md fájlban található.

```
int PWMrefresh=5500;
int PWMtiming[MAXBRIGHT+1] = {0,250,500,1000,2000,2500,3000,3500,4000,4500,5000};
```

Általános megjegyzés:
Mivel általában nem illik ilyen hosszú programokat megszakítás rutinba tenni, ezért a kritikus flash memória hozzáféréskor a ***EEPROMsaving*** változó jelzi a programban, hogy kicsit ne frissítsen, hogy ne zavarjon bele a flash időzítésbe. Ilyenkor egy pillanatra leáll a frissítés.

Letölthető: https://github.com/gpeter62/ESP_UniClock

További információ és segítségkérés, észrevétel, javaslat: gautier.p62@gmail.com

Készítette: Gautier Péter, 2021. január 23.

Felülvizsgálva és frissítve: 2026. szeptember, a 3.1-es firmware-verzióhoz (a driver-lista javítva, a SPIFFS adat-mappa elérési útja javítva, több új szakasz hozzáadva a jelenlegi kódbázis alapján, és mutató hozzáadva a projekt új docs/ fejlesztői dokumentációjához).

Minta óra beállítások a clocks.h fájlban:

Szabadon bővíthető az órák száma, mindenki elkészítheti a saját órájának megfelelő profilokat Szoftver frissítésnél nem kell belenyúlni az egyes program modulokba, az összes paraméter itt megadható és nem változik. Nyilván egy konkrét órára feltöltéskor mindig azt az órát kell engedélyezni, amit feltölteni akarunk. (Itt a példában a CLOCK_3 az.)

```
//#define CLOCK_1
#define CLOCK_3
//#define CLOCK_40
//#define CLOCK_41
//#define CLOCK_21

#ifdef CLOCK_1   //8266, UNFI PCB clock, 4x IN-16 tubes
  #define DEBUG
  #define USE_NEOPIXEL
  const byte tubePixels[] = {3,2,1,0};        //4 tubes, single leds, reverse direction
  //#define USE_DALLAS_TEMP
  #define TEMP_DALLAS_PIN -1    //DHT or Dallas temp sensor pin.  If not used, SET TO -1
  #define MULTIPLEX74141
  const byte digitEnablePins[] = {14,12,13,15};   //IN16 4x tube clock
  byte ABCDPins[4] =  {2,4,5,0};
  #define COLON_PIN   16        //Blinking Colon pin.  If not used, SET TO -1
  #define AP_NAME "UNICLOCK"
  #define AP_PASSWORD ""
  #define WEBNAME "IN-16 Nixie Clock"
#endif
```

```
#ifdef CLOCK_3   //8266, PCB less clock, IV-18 VFD tube
https://www.thingiverse.com/thing:3417955
  #define DEBUG
  #define USE_DALLAS_TEMP
  #define TEMP_SENSOR_PIN 4    //DHT or Dallas temp sensor pin.  If not used, SET TO -1
  #define MAX6921
  const byte segmentEnablePins[] =  {0,2,5,6,4,1,3,7};   //segment enable OUTbits of MAX6921
(a,b,c,d,e,f,g,DP)  (You MUST define always 8 Pins!!!)
  const byte digitEnablePins[] = {18,11,17,12,16,13,14,15}; //19};  //digit enable OUTbits of
MAX6921 1,2,3,4,5,6,7,8)  (You may define any number)
  //MAX6921 pins
    #define PIN_LE    12  // D6 Shift Register Latch Enable
    #define PIN_CLK   13  // D7 Shift Register Clock
    #define PIN_DATA  14  // D5 Shift Register Data
    #define PIN_BL    15  // D8 Shift Register Blank (1=display off     0=display on)
  #define AP_NAME "UNICLOCK"
  #define AP_PASSWORD ""
  #define WEBNAME "IV-18 VFD Clock"
#endif



//_______ESP-32 CLOCKS  (2x20pin Wemos D1 mini ESP32 modul) _____________
#ifdef CLOCK_40   //ESP32 WEMOS D1 mini, UNFI 6 x IV-11 VFD tubes clock
  #define DEBUG
  #define USE_NEOPIXEL
  #define NEOPIXEL_PIN 22
  byte tubePixels[] = {0,1,2,3,4,5};    //6 tubes, single leds
  //#define USE_DALLAS_TEMP
  #define USE_DHT_TEMP
  #define DHTTYPE DHT22
  #define TEMP_DHT_PIN 25    //DHT temp sensor pin.  If not used, SET TO -1
  #define MAX6921_ESP32
  const byte segmentEnablePins[] =  {19,17,15,12,13,16,18,14};   //segment enable OUTbits of
MAX6921 (a,b,c,d,e,f,g,DP)  (You MUST define always 8 Pins!!!)
  const byte digitEnablePins[] = {9,8,7,2,1,0};  //digit enable OUTbits of MAX6921 (1,2,3,4,5,6)
(You may define any number)
  #define DOUBLE_BLINK  //both separator points are blinking   //MAX6921 pins
    #define PIN_LE    4  // Shift Register Latch Enable
    #define PIN_CLK   17  // Shift Register Clock
    #define PIN_DATA  16  // Shift Register Data
    #define PIN_BL    32  // Shift Register Blank (1=display off     0=display on)
  #define ALARMSPEAKER_PIN 2   //Alarm buzzer pin
  #define ALARMBUTTON_PIN 0    //Alarm switch off button pin
  #define ALARM_ON HIGH         //How to switch ON alarm buzzer
  #define AP_NAME "UNICLOCK32"
  #define AP_PASSWORD ""
  #define WEBNAME "ESP32 IV-11 VFD-Clock"
#endif




#ifdef CLOCK_41   //ESP32 WEMOS D1 mini, UNFI board, 6 x Z573M Nixie tubes
  #define DEBUG
  #define USE_NEOPIXEL
  #define NEOPIXEL_PIN 22
  byte tubePixels[] = {0,1,2,3,4,5};    //6 tubes, single leds
  #define USE_DALLAS_TEMP
  //#define USE_DHT_TEMP
  //#define DHTTYPE DHT22
```

```
  //#define TEMP_DHT_PIN 25
  #define TEMP_DALLAS_PIN 25    //Dallas temp sensor pin.  If not used, SET TO -1
  #define MULTIPLEX74141_ESP32
  byte digitEnablePins[] = {26,18,33,19,23,5};   //ESP32 6x tube Clock
  byte ABCDPins[4] =  {32,16,4,17};
  #define DP_PIN 27             // decimalPoint inside Nixie tube, set -1, if not used!
  #define LEFTDECIMAL false   //set true (Z574M), if decimal point is on the left side on the
tube. Else set false (Z573M)!
  //#define TEMP_CHARCODE -1   //disable char => shift display right with 1 digit
  //#define GRAD_CHARCODE -1   //disable char => shift display right with 1 digit
  #define ALARMSPEAKER_PIN 2   //Alarm buzzer pin
  #define ALARMBUTTON_PIN 0    //Alarm switch off button pin
  #define ALARM_ON HIGH
  #define AP_NAME "UNICLOCK32"
  #define AP_PASSWORD ""
  #define WEBNAME "ESP32 Z573M Nixie-Clock"
#endif
```